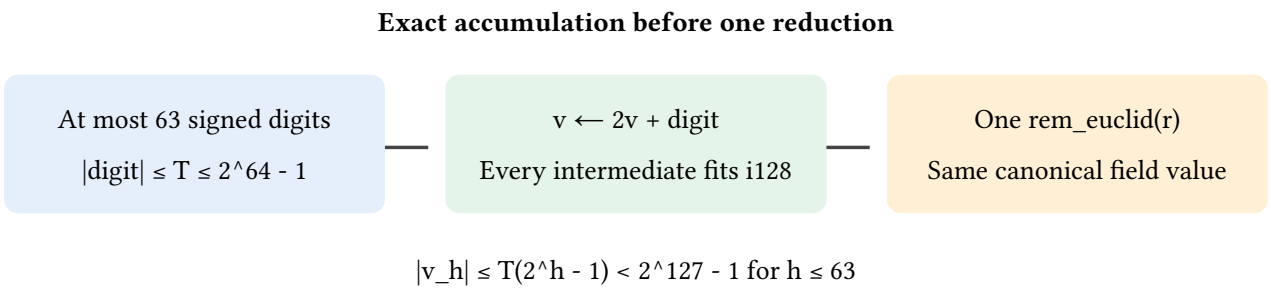
One-reduction bitplane accumulation

2026-11-05 · T-1584

The exact decoder now accumulates signed bitplane digits in i128 and reduces once at the end. A constructor-based range proof covers each doubling and addition; the source, coefficient packet, bitplanes and recovery code are unchanged.



Range argument

After h digits, the accumulator magnitude is at most T(2^h-1). Here T is at most2^64-1 and h is at most63. The maximum is2^127-2^64-2^63+1, strictly below i128::MAX. The intermediate doubling before adding a digit also fits. A compile-time assertion excludes pointer widths above64. Euclidean reduction at the end gives the same canonical residue as reduction at every step.

Checked implementation

One optimized build with overflow-checks=yes and one bounded invocation pass all283 native methods.1024 synthetic extreme digit vectors are checked against an independently ordered checked weighted sum;4096 comparisons match the previous modular fold. Positive and negative maximum bounds fit. All479 saved polynomial inputs and all8 full recovery receipts remain correct.36 prebuild source/data locks verify;19 compiler warnings are retained. Synthetic extreme term counts test arithmetic only, without allocating those terms.

Scope and next action

For the saved nine-bit coefficient packet, reductions fall from9 to1 by source derivation. Nine signed Horner steps,144 population-count calls,328 stored plane words and constructor transposition remain. No timing comparison or cloud job ran. Next include this variant in complete decoder/setup accounting. Coefficient-program construction and original4563-case cohort remain separate preserved requirements.

Evidence

DERIVATION.md, control.json, source-lock.json, receipt.json and raw logs. Type/remainder semantics: Rust i128 documentation, read before the interruption. Prior context: the checked bitplane decoder. This is an application of standard bounded exact accumulation.